

FlightSound

Real-time aircraft noise awareness for quieter planning near airports



CS3337 • Product Presentation & Demo

Team members: Agustin, Angel, Carter, Daphne, Isaac, Kassandra, Ruby, and Tiffany

GitHub: [Group 06](#) | App: local demo at <https://localhost:5500/>



Customer Vision: problem → digital solution

For residents/students near airports.

Problem

People living near airports often experience disruptive aircraft noise that can interfere with daily life, while having little way to predict when loud flight activity will occur.



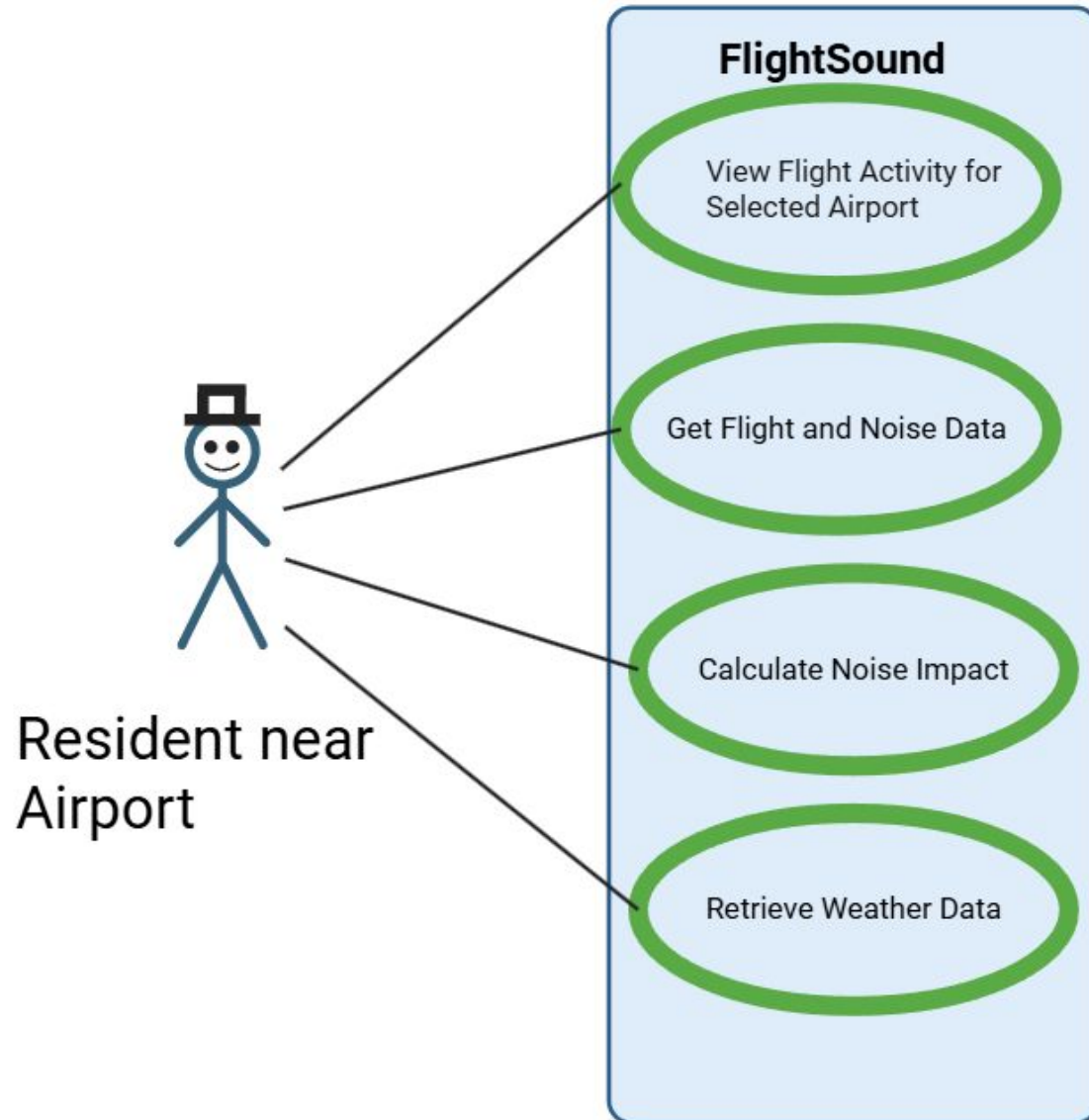
Digital Solution

FlightSound is a web-based platform that displays live flights near airports on Google Maps, estimates aircraft noise levels using flight data, and color-codes noise impact to help users identify quieter times.

Vision: “Our vision is to give people living near airports an easy way to understand and plan around aircraft noise using real-time flight data.”

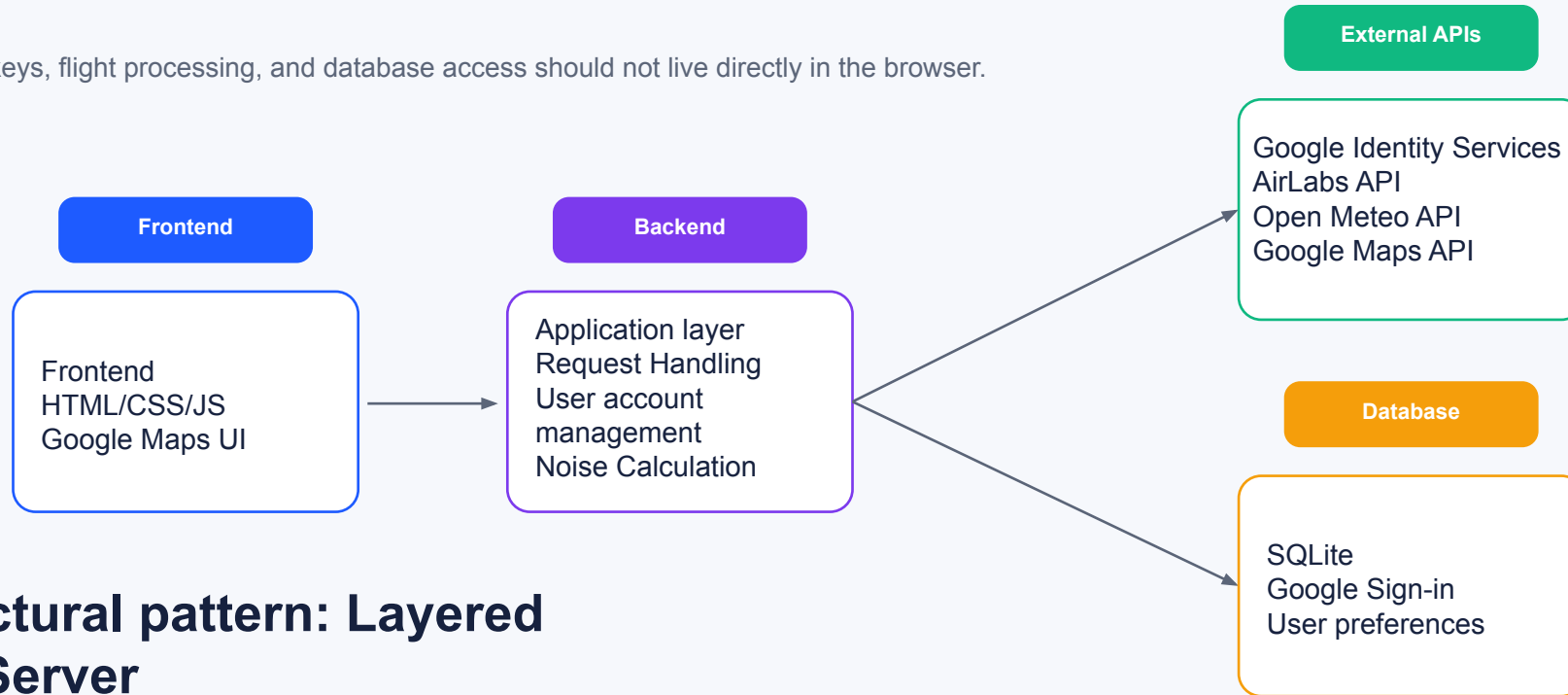
Use case diagram

Main users interact with search, map



Architectural Design: Layered Client-Server Pattern

Chosen because API keys, flight processing, and database access should not live directly in the browser.



Architectural pattern: Layered Client-Server

FlightSound separates the system into four components: frontend, backend, database, and external APIs.

This layered architecture was chosen to make each subsystem independently testable and to allow the team to work on components in parallel.

Non-functional requirements behind this choice

Maintainability: parts can change independently.

Security: keys and data are not exposed in client code.

Usability: frontend stays responsive and visual.

Reliability: Java's strong static typing catches errors at compile time rather than runtime,

Detailed Design: Technologies Used

Each technology supports one part of the final working system.

Frontend

HTML/CSS/JavaScript
Builds the interactive
web interface and
sidebar.

Google Maps JavaScript
API, shows airport,
aircraft markers, noise
circles, and map controls.

Backend

Java HTTP Server
Handles /api/noise
requests and returns
JSON.

Gson + JSON
Parses API data and
sends structured data to
the frontend.

Data & APIs

AirLabs + Open Meteo API
Provides live
flight/weather data used
by the backend.

SQLite
Stores user accounts,
usernames, and saved
airport/location.

Google Sign-In
Allows account-based
preferences.

Tools

Kiro + VS Code + GitHub
AI-assisted
development, local
testing, version control.

Local demo setup: frontend + Java backend run on the developer machine, usually through localhost:8080.

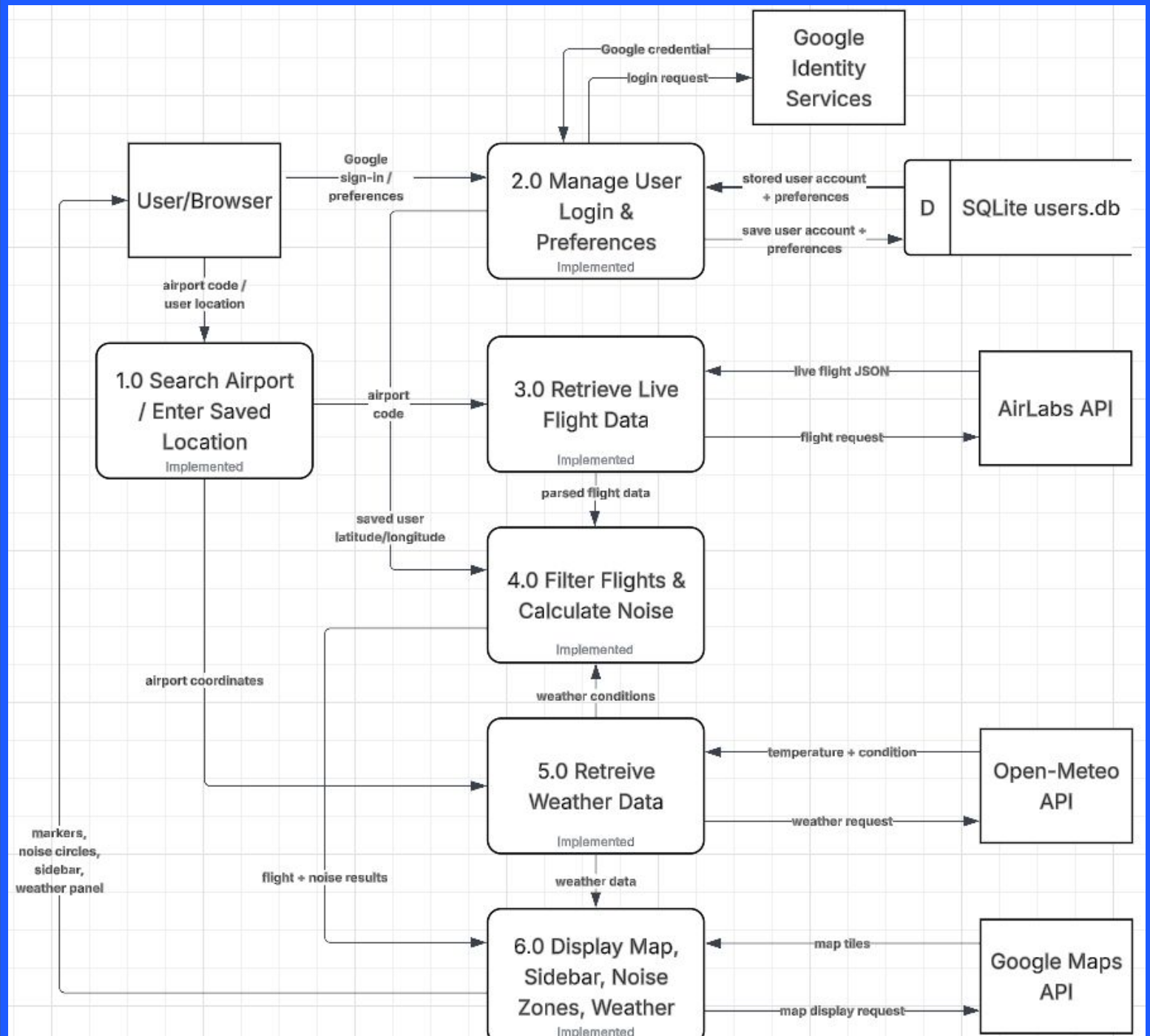
Detailed Design: Level 1 Data Flow Diagram

Legend

- External Entity
- Process
- D Data Store
- Data Flow

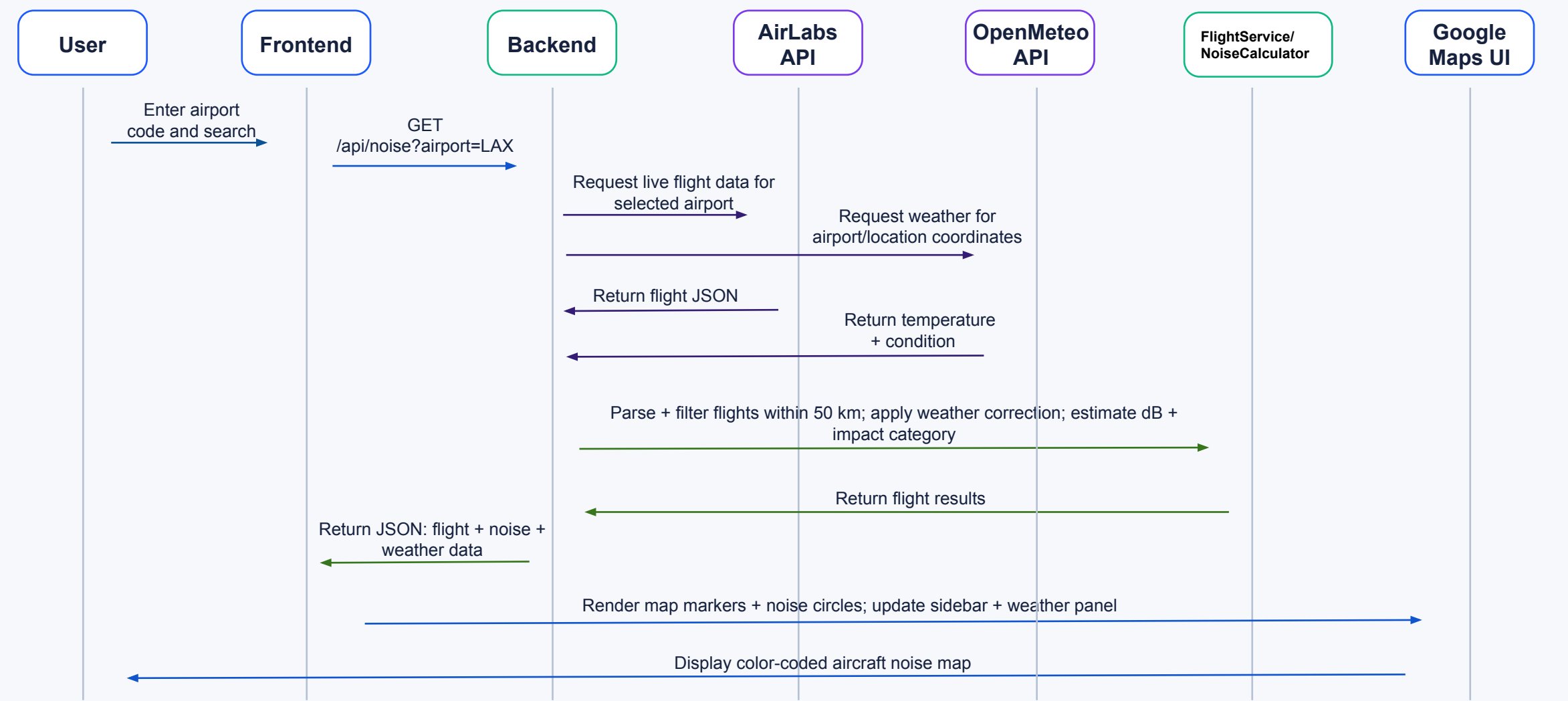
Main Output to UI

- Flight number
- Aircraft type
- Altitude
- Estimated dB
- impact category
- coordinates
- Marker color
- weather data



Detailed Design: Sequence Diagram of Feature Process

Featured Process: Search Airport and Display Aircraft Noise



Result: The system processes raw flight data into a visual, color-coded noise map for the user.

Lessons Learned: Agile + New Technologies

Focus on what changed during development and what the team learned.

What went well

- Integrated flight API with map UI
- Connected frontend and Java backend
- Improved UI based on testing
- Used Git branches and pull requests

What we improved

- Replaced unclear placeholders
- Fixed location/map viewport issues
- Added login/preferences flow
- Kept scope realistic near the end

What we learned

- APIs need careful error handling
- “Working locally” is different from deploying
- Noise modeling needs location filtering
- Agile demos reveal usability gaps fast

PI Roadmap

Summarize current progress and next steps by PI.

PI 26.1 Foundation

Frontend layout
Google Maps
display
Initial AirLabs
connection

PI 26.2 Core Product

Live airport
search
Noise levels +
categories
Login +
preferences
Local demo

PI 26.3 Refinement

Geofence flights
by radius
Improve
quiet-time
algorithm
Real weather
integration
Better dB
comparisons

Future Enhancements

Public
deployment
User
dashboards
Historical trends
Predictive flight
paths

Live Demo

<http://localhost:5500/>

1

Start backend + open app

2

Airport Search (LAX as example)

3

Explain markers (click on flights)

4

Show preferences (Login & Save Preferences)

5

Show flights based on Airport and User Location



Q&A

Thank you!

